
Chapter 10 ArrayLists

10.1 ArrayLists

- **Basic ArrayList Operations** 
- **ArrayList Searching Methods** 
- **A Complete ArrayList Program** 
- **Adding to and Removing from an ArrayList** 
- **Initializing an ArrayList** 
- **Using the For-Each Loop with ArrayLists** 
- **Wrapper Classes** 

10.2 The Comparable Interface

- **Natural Ordering and compareTo** 
- **Implementing the Comparable Interface** 

10.3 Case Study: Ranked Choice Voting

- **Ballot Class** 

- Counting Votes 
- Multiple Rounds 

Introduction

*One of the most fundamental data structures you will encounter in programming is a list. You'll want to store lists of words, lists of numbers, lists of names, and so on. **Chapter 7**  demonstrated using arrays to store sequences of values, but arrays are fixed-size structures that require you to declare in advance exactly how many elements you want to store. In this chapter we'll explore a new structure, known as an `ArrayList`, that provides more flexibility than an array. An `ArrayList` is a dynamic structure with a variable length, so it can grow and shrink as the program executes.*

The `ArrayList` structure is the first example we have discussed of a generic structure that can be used to store values of different types. As a result, we will need to explore some issues related to generic structures in this chapter. We will also look at how to use primitive data with such structures, using what are known as the

wrapper classes. Finally, we will demonstrate how to use the `Comparable` interface to put values of a particular type into sorted order and how to write classes that implement the `Comparable` interface.

10.1 ArrayLists

In our daily lives, we often manipulate lists of one kind or another. For example, on social networking sites like Facebook.com, people list the bands they like. Suppose someone listed the following bands:

```
Tool, U2, Phish, Pink Floyd, Radiohead
```

You saw in [Chapter 7](#) that you can declare an array to store a sequence of values. For example, to store the preceding list, you could declare an array of `String`s of length 5. But what happens if you want to change the list later (say, to remove Tool and U2 from the list)? You would have to shift values over, and you'd be left with empty array slots at the end. And what if you wanted to add to the list, so that it ended up with more than five names? You wouldn't have room to store more than five values in the original array, so you would have to construct a new array with a larger size to store the list.

Most of us think of lists as being more flexible than that. We don't want to have to worry about the kind of low-level details that come up in the course of manipulating an array. We want to be able to just say, "Add something to the list" or "Remove this value from the list," and we want the lists to grow and shrink over time as we add or

remove values. Computer scientists would say that we have in mind a *list abstraction* that enables us to specify certain operations to be performed (add, remove) without having to worry about the details of how those operations are performed (shifting, constructing new arrays).

Java provides this functionality in a class called `ArrayList`. Internally, each `ArrayList` object uses an array to store its values. As a result, an `ArrayList` provides the same fast random access as an array. But unlike with an array, with an `ArrayList` you can make simple requests to add or remove values, and the `ArrayList` takes care of all of the details for you: If you add values to the list it makes the array bigger, and if you remove values it handles any shifting that needs to be done.

Remember that you can declare arrays of different types. If you want an array of `int` values, you declare a variable of type `int[]`. For an array of `String` values, you use the type `String[]`. This is a special syntax that works just for arrays, but the `ArrayList` class has almost the same flexibility. If you read the API documentation for `ArrayList`, you'll see that it is actually listed as `ArrayList<E>`. This is an example of a *generic class* in Java.

Generic Class (Generic)

A class such as `ArrayList<E>` that takes a type parameter to indicate what kind of values it will use.

The “E” in `ArrayList<E>` is short for “Element,” and it indicates the type of elements that will be included in the `ArrayList`. Generic classes are similar to parameterized methods. Remember from [Chapter 3](#)  that you can use a parameter to define a family of related tasks that differ just by a particular characteristic like height or width. In this case, the parameter is a type and it is used to declare another type. The type `ArrayList<E>` represents a family of types that differ just by the type of element they store. You would use `ArrayList<String>` to store a list of `Strings`, `ArrayList<Point>` to store a list of `Points`, `ArrayList<Color>` to store a list of `Colors`, and so on. Notice that you would never actually declare something to be of type `ArrayList<E>`. As with any parameter, you have to replace the `E` with a specific value to make it clear which of the many possible `ArrayList` types you are using.

Basic `ArrayList` Operations

The `ArrayList` class is part of the `java.util` package, so to include it in a program, you must include an `import` declaration:

```
import java.util.*;    // for ArrayList
```

The syntax for constructing an `ArrayList` is more complicated than what we've seen before because of the required type parameter. For example, you could construct an `ArrayList` of `String`s as follows:

```
// construct a list of strings (long form)
ArrayList<String> list = new ArrayList<String>();
```

This code constructs an empty list of strings. The syntax is complicated, but it will be easier to remember if you keep in mind that the `<String>` notation is actually part of the type: This isn't simply an `ArrayList`, it is an `ArrayList<String>`, often read as “an `ArrayList` of `String`.” Notice how the type appears on both sides of the `=` sign, when you declare the variable and when you call the constructor.

If you think in terms of the type being `ArrayList<String>`, you'll see that this line of code isn't all that different from the code used to construct an object like a `Point`. In both cases, you write the variable's type on the left side of the `=` sign before its name and after the `=` sign and the keyword `new` (which calls that type's constructor to create an object of that type):

```
// construct a normal object  
Point p = new Point();
```

It can be cumbersome to list the element type `<String>` twice, so Java version 7 introduced a new shorter syntax for declaring collections called the “diamond operator” whereby the element type may be omitted on the right side of the statement and replaced by `<>`, such as:

```
// construct a list of strings (short form with diamond  
operator)  
ArrayList<String> list = new ArrayList<>();
```

The diamond operator syntax is shorter and more convenient, and its behavior is identical to that of the longer code. So we will use the short form for the rest of our examples in this text.

In the rest of this section we’ll manipulate a list of a person’s favorite music groups represented as strings. Once you’ve constructed an `ArrayList`, you can add values to it by calling its `add` method:

```
// add values to a list
```

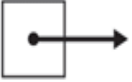
```
ArrayList<String> bands = new ArrayList<>();  
bands.add("Tool");  
bands.add("Phish");  
bands.add("Pink Floyd");
```

When you ask an `ArrayList` to add a new value to the list, it appends the new value to the end of the list. Unlike with a simple array, an `ArrayList` can be thought of as having a dynamic size that grows and shrinks to fit the elements you put inside it. So you can think of the `add` method as performing two major tasks: expanding the list's size by one to accommodate the new element, and placing the new element at the end of the list. **Figure 10.1**  shows the state of our list of bands after each of the preceding three elements is added:

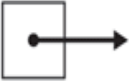
Figure 10.1 Adding elements to a list

bands  [empty]

```
bands.add("Tool");
```

bands   [0]

```
bands.add("Phish");
```

bands   [0]  [1]

```
bands.add("Pink Floyd");
```

bands   [0]  [1]  [2]

Java will make sure that you add values of an appropriate type. In this case, because you requested an `ArrayList<String>`, you can add `String` elements to the list. If you try to add a value of a different type, such as an integer or `Point`, the code will generate a compiler error.

Unlike with simple arrays, printing an `ArrayList` is straightforward because the `ArrayList` class overrides Java's `toString` method. The `ArrayList` version of `toString` constructs a string that includes the contents of the list inside square brackets with the values separated by commas. Remember that the `toString` method is called when you

print an object or concatenate an object to a string. As a result, you can print an `ArrayList` with a simple `println` statement:

```
System.out.println("My fave bands: " + bands);
```

For example, you can add `println` statements as you add each element to the list. Or if you are testing `ArrayList` features in JShell, you can just type the list's name to print its contents, as in the following interaction:

```
jshell> ArrayList<String> bands = new ArrayList<>();
```

```
bands ==> []
```

```
jshell> bands.add("Tool");
```

```
jshell> bands
```

```
bands ==> [Tool]
```

```
jshell> bands.add("Phish");
```

```
jshell> bands
```

```
bands ==> [Tool, Phish]
```

```
jshell> bands.add("Pink Floyd");
```

```
jshell> bands  
bands ==> [Tool, Phish, Pink Floyd]
```

Notice that you can print the list even when it is empty and each time that new values are added to the end of the list. The `ArrayList` class also provides an overloaded version of the `add` method for adding a value at a particular index in the list. It preserves the order of the other list elements, shifting values to the right to make room for the new value. This version of `add` takes two parameters: an index and a value to insert. `ArrayList`s use zero-based indexing, just as arrays and strings do. For example, given the preceding list, consider the effect of inserting a value at index 1:

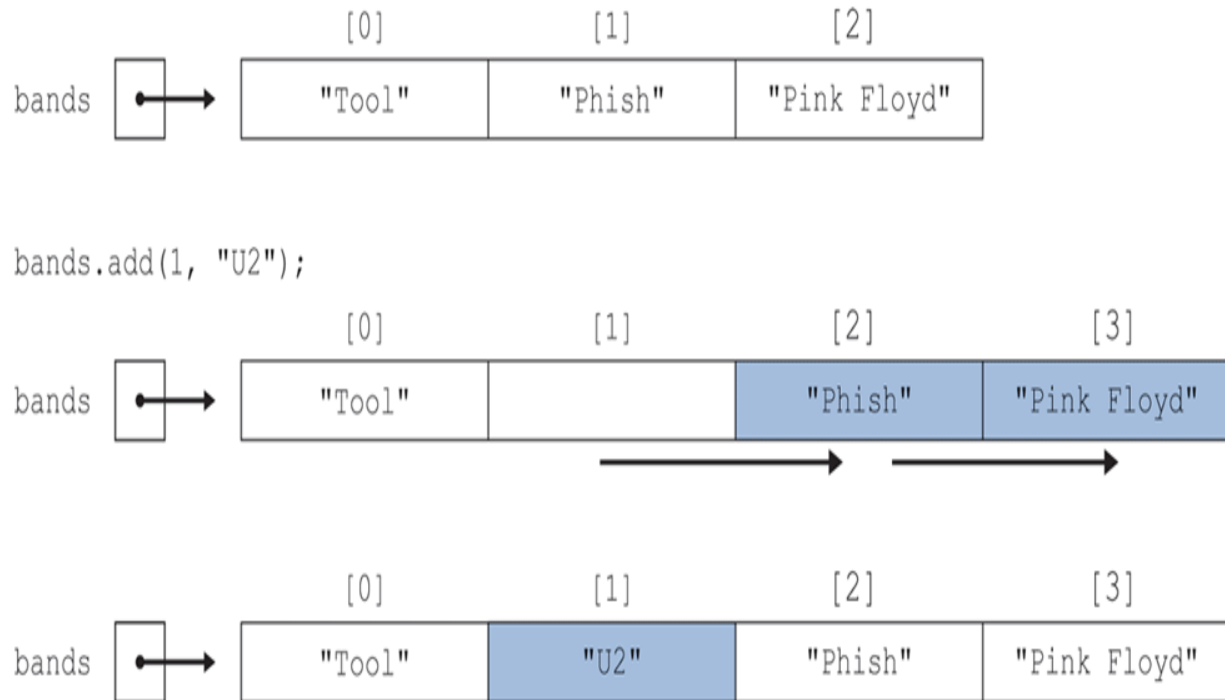
```
jshell> bands  
bands ==> [Tool, Phish, Pink Floyd]  
  
jshell> bands.add(1, "U2");  
  
jshell> bands  
bands ==> [Tool, U2, Phish, Pink Floyd]
```

The call on `add` instructs the list to insert the new element at index 1. Therefore, the old value at index 1 and everything that follows it gets shifted to the right. You can think of this method as performing three

operations: expanding the list's size by 1, shifting elements right to make room for a new element, and inserting the new element.

Figure 10.2  shows the state of our list before and after inserting U2 at index 1.

Figure 10.2 Inserting an element at a given index



The `ArrayList` also has a `remove` method that accepts an integer index as a parameter and removes the value at that index. The method preserves the order of the list by shifting values to the left to fill in any gap. For example, consider what happens to the previous list if we remove the value at position 0 and then remove the value at position 1:

```
jshell> System.out.println("before remove bands = " + bands);
```

```
before remove bands = [Tool, U2, Phish, Pink Floyd]
```

```
jshell> bands.remove(0);
```

```
jshell> bands.remove(1);
```

```
jshell> System.out.println("after remove bands = " + bands);
```

```
after remove bands = [U2, Pink Floyd]
```

This result is a little surprising. We asked the list to remove the value at position 0 and then to remove the value at position 1. You might imagine that this would get rid of the strings `"Tool"` and `"U2"`, since they were at positions 0 and 1, respectively, before this code was executed. However, an `ArrayList` is a dynamic structure whose values move around and shift into new positions in response to your commands. The order of events is demonstrated more clearly if we include a third `println` statement in the code:

```
jshell> System.out.println("before remove bands = " + bands);
```

```
before remove bands = [Tool, U2, Phish, Pink Floyd]
```

```
jshell> bands.remove(0);
```

```
jshell> System.out.println("after 1st remove bands = " +  
bands);  
after 1st remove bands = [U2, Phish, Pink Floyd]  
  
jshell> bands.remove(1);  
  
jshell> System.out.println("after 2nd remove bands = " +  
bands);  
after 2nd remove bands = [U2, Pink Floyd]
```

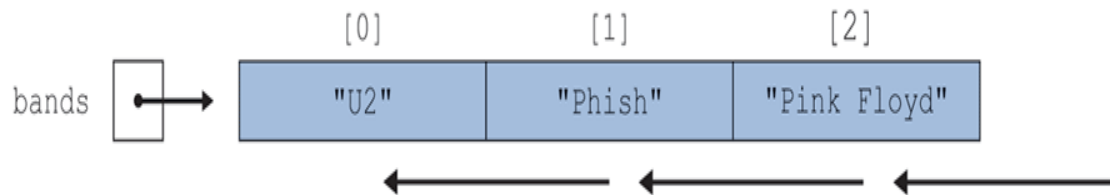
The first call on `remove` removes `"Tool"` because it's the value currently in position 0. But once that value has been removed, everything else shifts over: The element `"U2"` moves to the front (to position 0), the element `"Phish"` shifts into position 1, and the element `"Pink Floyd"` moves into position 2. So, when the second call on `remove` is performed, Java removes `"Phish"` from the list because it is the value that is in position 1 at that point in time.

Figure 10.3  depicts the two removals from the list.

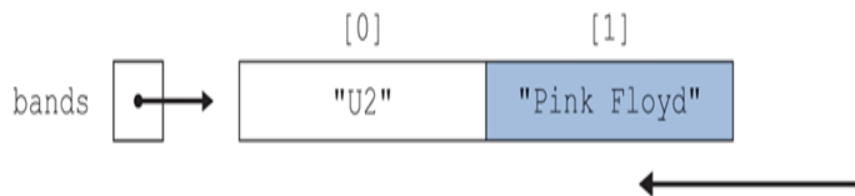
Figure 10.3 Removing an element from a given index



```
bands.remove(0);
```



```
bands.remove(1);
```



If you want to find out the number of elements in an `ArrayList`, you can call its `size` method. If you want to obtain an individual item from the list, you can call its `get` method, passing it a specific index. These two methods are often used together. For example, the following loop would add up the lengths of the strings in the preceding list of bands:

```
// loop over the elements of a list
int sum = 0;
for (int i = 0; i < bands.size(); i++) {
    String s = bands.get(i);
```

```
        sum += s.length();  
    }  
    System.out.println("Total of lengths = " + sum);
```

This loop looks similar to the kind of loop you would use to access the various elements of an array, but instead of asking for the `bands.length` as you would for an array, you ask for `bands.size()`, and instead of asking for `bands[i]` as you would with an array, you ask for `bands.get(i)`.

Calling `add` and `remove` can be expensive in terms of time because the computer has to shift the values around. If all you want to do is to replace a value, you can use the list's `set` method, which takes an index and a value and replaces the value at the given index with the given value without doing any shifting. For example, you could replace the value at the front of the sample list by writing the following line of code:

```
// modify an existing element value  
bands.set(0, "The Violent Femmes");
```

As noted earlier, when you construct an `ArrayList` it will initially be empty. After you have added values to a list, you can remove them one at a time. But what if you want to remove all of the values from the list? In that case, you can call the list's `clear` method.

```
// remove all element values  
bands.clear();
```

Table 10.1  summarizes the `ArrayList` operations introduced in this section. A more complete list can be found in the online Java documentation.

Table 10.1 Basic `ArrayList` Methods

`ArrayList` Searching Methods

Once you have built up an `ArrayList`, you might be interested in searching for a specific value in the list. The `ArrayList` class provides several mechanisms for doing so. If you just want to know whether or not something is in the list, you can call the `contains` method, which returns a Boolean value. For example, suppose you have an input file of names that has some duplicates, and you want to get rid of the duplicates. The file might look like this:

```
Maria Derek Erica  
Livia Jack Anita  
Kendall Maria Livia Derek
```

Jamie Jack

Erica

You can construct an `ArrayList<String>` to hold these names and use the `contains` method to ensure that there are no duplicates:

```
// removes duplicates from a list
Scanner input = new Scanner(new File("names.txt"));
ArrayList<String> list = new ArrayList<>();
while (input.hasNext()) {
    String name = input.next();
    if (!list.contains(name)) {
        list.add(name);
    }
}
System.out.println("list = " + list);
```

Given the sample input file, this code produces the following output:

```
list = [Maria, Derek, Erica, Livia, Jack, Anita, Kendall, Jamie]
```

Notice that only 8 of the original 13 names appear in this list, because the various duplicates have been eliminated.

Sometimes it is not enough to know that a value appears in the list. You may want to know exactly where it occurs. For example, suppose you want to write a method to replace the first occurrence of one word in an `ArrayList<String>` with another word. You can call the `set` method to replace the value, but you have to know where it appears in the list. You can find out the location of a value in the list by calling the `indexOf` method.

The `indexOf` method takes a particular value and returns the index of the first occurrence of the value in the list. If it doesn't find the value, it returns `-1`. So, you could write a `replace` method as follows:

```
public static void replace(ArrayList<String> list,
                           String target, String replacement) {
    int index = list.indexOf(target);
    if (index >= 0) {
        list.set(index, replacement);
    }
}
```

Notice that the return type of this method is `void`, even though it changes the contents of an `ArrayList` object. Some novices think that you have to return the changed `ArrayList`, but the method doesn't actually create a new `ArrayList`; it merely changes the contents of the list. As you've seen with arrays and other objects, a

parameter is all you need to be able to change the current state of an object because objects involve reference semantics in which the method is passed a reference to the object.

You can test the method with the following code:

```
ArrayList<String> list = new ArrayList<>();  
list.add("to");  
list.add("be");  
list.add("or");  
list.add("not");  
list.add("to");  
list.add("be");  
System.out.println("initial list = " + list);  
replace(list, "be", "beep");  
System.out.println("final list  = " + list);
```

This code produces the following output:

```
initial list = [to, be, or, not, to, be]  
final list   = [to, beep, or, not, to, be]
```

There is also a variation of `indexOf` known as `lastIndexOf`. As its name implies, this method returns the index of the last occurrence of a value. There are many situations where you might be more interested in the last occurrence rather than the first occurrence. For example, if a bank finds a broken automated teller machine, it might want to find out the name and account number of the last customer to use that machine. **Table 10.2**  summarizes the `ArrayList` searching methods.

Table 10.2 `ArrayList` Searching Methods

All of the `ArrayList` searching methods call the `equals` method for comparing values. The method names are fairly standard and appear elsewhere in the Java class libraries. For example, the `String` class also has methods called `indexOf` and `lastIndexOf` that allow you to search for the position of a character or substring inside a string.

A Complete `ArrayList` Program

Before we go further, let's look at a complete `ArrayList` program. Search engines like Google ignore the *stop words* in users' queries. The idea is that certain words like "a" and "the" appear so often that they aren't worth indexing. Google won't disclose the exact list of words it uses, although a few examples are listed on the web site and people have speculated about what they think is on the list.

Google's full list of stop words is believed to have at least 35 entries, but we'll settle for 15 of the most obvious choices. To explore how removing stop words can affect a text, our program will read a file called `speech.txt` that contains the first part of Hamlet's famous speech:

```
To be or not to be - that is the question:  
Whether 'tis nobler in the mind to suffer  
The slings and arrows of outrageous fortune  
Or to take arms against a sea of troubles,  
And by opposing end them.
```

The program constructs a list of stop words and then reads the file word by word, printing every word that is not a stop word. To avoid issues of case, the stop words are all in lowercase and the call on `contains` is passed a lowercase version of each word from the input file. Here is the complete program:

```
1 // This program constructs a list of stop words and echoes  
2 // Hamlet's famous speech with the stop words removed.  
3  
4 import java.util.*;  
5 import java.io.*;  
6
```

```
7 public class StopWords {
8     public static void main(String[] args)
9         throws FileNotFoundException {
10         // build the list of stop words
11         ArrayList<String> stopWords = new ArrayList<>();
12         stopWords.add("a");
13         stopWords.add("be");
14         stopWords.add("by");
15         stopWords.add("how");
16         stopWords.add("in");
17         stopWords.add("is");
18         stopWords.add("it");
19         stopWords.add("of");
20         stopWords.add("on");
21         stopWords.add("or");
22         stopWords.add("that");
23         stopWords.add("the");
24         stopWords.add("this");
25         stopWords.add("to");
26         stopWords.add("why");
27
28         // process the file, printing all but stop words
29         Scanner input = new Scanner(new
File("speech.txt"));
30         while (input.hasNext()) {
31             String next = input.next();
32             if (!stopWords.contains(next.toLowerCase())) {
33                 System.out.print(next + " ");
```

```
34     }  
35  
36     }  
37 }
```

The program produces the following output:

```
not - question: Whether 'tis nobler mind suffer slings and  
arrows outrageous fortune take arms against sea troubles,  
And opposing end them.
```

This output represents the search view of the original text (the core set of words that will be used by a search engine).

Adding to and Removing from an

`ArrayList`





In this section, we will explore some of the issues that come up when you dynamically add values to or remove values from the middle of an `ArrayList`. The results are often surprising, so it is worth exploring the common pitfalls.

Consider the following code, which creates an `ArrayList` and stores several words in it:

```
ArrayList<String> words = new ArrayList<>();  
words.add("four");  
words.add("score");  
words.add("and");  
words.add("seven");  
words.add("years");  
words.add("ago");  
System.out.println("words = " + words);
```

This code produces the following output:

```
words = [four, score, and, seven, years, ago]
```

We'll explore the problem of inserting a tilde ("~") in front of each word, doubling the size of the list. Inserting tildes isn't the most exciting operation you can imagine doing with a list, but we want to keep things simple so we can focus on the programming issues, and you'll find that you often want to perform operations like this. For example, if you put a tilde in front of a search term, Google does a different search that includes synonyms of the word. Searching for "~four ~score" yields more than 10 times as many pages as searching for just "four score."

In our case, we want to keep the tildes separate from the words themselves, so we want to insert a new `String` containing just a tilde in front of each word in the list. Here is a first attempt that makes sense intuitively but doesn't work:

```
// doesn't work properly
for (int i = 0; i < words.size(); i++) {
    words.add(i, "~");
}
System.out.println("after loop words = " + words);
```

This `for` loop is a slight variation of the standard array-traversal loop. It has an index variable `i` whose value starts at `0` and goes up by one each time. In this case, the loop is inserting a tilde at position

`i` each time it executes the loop. The problem is that the loop never terminates. (If you're patient enough, you will find that the program does eventually terminate with an "out of memory" error.)

The loop fails to terminate because the `ArrayList` structure is dynamic in nature. Let's think about this carefully to see what's happening. Initially we have the following list, with the `String` "four" in position 0:

```
[four, score, and, seven, years, ago]
```

The first time the program executes the loop, it inserts a tilde at position 0. To make room for the tilde at position 0, the `ArrayList` has to shift all the other values one place to the right. As a result, the `String` "four" ends up in position 1:

```
[~, four, score, and, seven, years, ago]
```

Then we come around the `for` loop, increment `i` to be 1, and insert a tilde at position 1. But because the word "four" is currently at position 1, this second tilde also goes in front of the word "four", shifting it into position 2:

```
[~, ~, four, score, and, seven, years, ago]
```

We then go around the loop again, incrementing `i` to be `2` and inserting a tilde at that position, which is once again in front of the word "`four`":

```
[~, ~, ~, four, score, and, seven, years, ago]
```

This loop continues indefinitely, because we keep inserting tildes in front of the first word in the list. The `for` loop test compares `i` to the size of the list, but because the list is growing, the size keeps going up. So, this process continues until all the computer's available memory is exhausted.

To fix this loop, we have to take into account the fact that inserting a tilde at position `i` is going to shift everything one place to the right. So, on the next iteration of the loop, we will want to insert the tilde in the position that is two to the right, not one to the right. We can fix the code simply by changing the update part of the `for` loop to add 2 to `i` instead of adding 1 to `i`:

```
for (int i = 0; i < words.size(); i += 2) {  
    words.add(i, "~");  
}  
System.out.println("after loop words = " + words);
```

When we execute this version of the code, we get the following output:

```
after loop words = [~, four, ~, score, ~, and, ~, seven, ~,  
years, ~, ago]
```

As another example, let's consider what code we would need to write to undo this operation. We want to write code that will remove every other value from the list, starting with the first value—in other words, the values that are currently at indexes 0, 2, 4, 6, 8, and 10. We might write code like the following:

```
// doesn't work properly  
for (int i = 0; i < words.size(); i += 2) {  
    words.remove(i);  
}
```

```
}  
System.out.println("after second loop words = " + words);
```

Looking at the loop, you can see that `i` starts at `0` and goes up by 2 each time, which means it produces a sequence of even values (`0`, `2`, `4`, and so on). That seems to be what we want, given that the values to be removed are at those indexes. But this code doesn't work. It produces the following output:

```
after second loop words = [four, ~, ~, and, seven, ~, ~, ago]
```

Again, the problem comes from the fact that in the `ArrayList` values are shifted dynamically from one location to another. The first tilde we want to remove is at index 0:

```
[~, four, ~, score, ~, and, ~, seven, ~, years, ~, ago]
```

But once we remove the tilde at position 0, everything is shifted one position to the left. The second tilde moves into index 1:

```
[four, ~, score, ~, and, ~, seven, ~, years, ~, ago]
```

So the second remove should be at index 1, not index 2. And once we perform that second remove, the third tilde will be in index 2:

```
[four, score, ~, and, ~, seven, ~, years, ~, ago]
```

In this case, we don't want to increment `i` by 2 each time through the loop. Here, the simple loop that increments by 1 is the right choice:

```
for (int i = 0; i < words.size(); i++) {  
    words.remove(i);  
}  
  
System.out.println("after second loop words = " + words);
```

After the program executes this code, it produces the following output:

```
after second loop words = [four, score, and, seven, years, ago]
```

Putting all of these pieces together gives us the following complete program:

```
1  // Builds up a list of words, adds tildes, and removes
them.
2
3  import java.util.*;
4
5  public class TildeFun {
6      public static void main(String[] args) {
7          // construct and fill up ArrayList
8          ArrayList<String> words = new ArrayList<>();
9          words.add("four");
10         words.add("score");
11         words.add("and");
12         words.add("seven");
13         words.add("years");
14         words.add("ago");
15         System.out.println("words = " + words);
16
17         // insert one tilde in front of each word
18         for (int i = 0; i < words.size(); i += 2) {
19             words.add(i, "~");
```

```

20     }
21     System.out.println("after loop words = " + words);
22
23     // remove tildes
24     for (int i = 0; i < words.size(); i++) {
25         words.remove(i);
26     }
27     System.out.println("after second loop words = " +
words);
28 }
29 }

```

If we want to write the loops in a more intuitive manner, we can run them backwards. The loops we have written go from left to right, from the beginning of the list to the end of the list. We could instead go from right to left, from the end of the list to the beginning of the list. By going backward, we ensure that any changes we are making occur in parts of the list that we have already visited.

For example, we found that the following loop did not work properly even though it seemed like the intuitive approach:

```

// doesn't work properly
for (int i = 0; i < words.size(); i++) {
    words.add(i, "~");
}

```

But if we turn this loop around and have it iterate backward rather than going forward, it does work properly:

```
// works properly because loop goes backwards
for (int i = words.size() - 1; i >= 0; i--) {
    words.add(i, "~");
}
```

The problem with the original code was that we were inserting a value into the list and then moving our index variable onto that spot in the list. If instead we work backward, the changes that we make affect only those parts of the list that we have already processed.

Similarly, we tried to write the second loop as follows:

```
// doesn't work properly
for (int i = 0; i < words.size(); i += 2) {
    words.remove(i);
}
```

Again, the problem was that we were changing a part of the list that we were about to process. We can keep the overall structure intact by running the loop backward:

```
// works properly because loop goes backwards
for (int i = words.size() - 2; i >= 0; i -= 2) {
    words.remove(i);
}
```

Initializing an `ArrayList`

In [Chapter 7](#)  we saw different ways to construct arrays and initialize the values of each element of an array. We saw the typical syntax for creating an array of a given length and setting each of its elements was the following:

```
// create an array of names of days of the week
String[] dayNames = new String[7];
dayNames[0] = "Mon";
dayNames[1] = "Tue";
dayNames[2] = "Wed";
dayNames[3] = "Thu";
dayNames[4] = "Fri";
dayNames[5] = "Sat";
dayNames[6] = "Sun";
```

If we wanted an `ArrayList` of these elements rather than an array, the corresponding code would be the following:

```
// create an ArrayList of names of days of the week
ArrayList<String> dayNames = new ArrayList<>();
dayNames.add("Mon");
dayNames.add("Tue");
dayNames.add("Wed");
dayNames.add("Thu");
dayNames.add("Fri");
dayNames.add("Sat");
dayNames.add("Sun");
```

It can be tedious to set up an array or list of values in the preceding way because it takes so many lines to store the element values. In **Chapter 7**  we also showed a syntax for quickly initializing an array from a given list of element values:

```
// create an array of names of days of the week (shorter)
String[] dayNames = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun"};
```

Unfortunately the `ArrayList` does not support the `{}` syntax we used with arrays. But there is a static method called `List.of` that accepts any number of element values as parameters and returns a list object containing those elements. We can then pass this to the constructor of an `ArrayList` to quickly initialize the `ArrayList` to store the values we want:

```
// create an ArrayList of names of days of the week (shorter)
ArrayList<String> dayNames = new ArrayList<>(
    List.of("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"));
```

Previous versions of Java used a method called `Arrays.asList` that behaves much the same way as `List.of`. But as of Java 8 the latter method is preferred, partly because it is shorter to type and partly because it is more consistent with methods for creating other types of collections. In the next chapter we will learn about collections called sets and maps, and those collection types have methods called `Set.of` and `Map.of` that help you create collections. Using a consistent convention like this makes the method names easier to remember.

Using the For-Each Loop with `ArrayList`s

You saw in [Chapter 7](#) that you can use a for-each loop to iterate over the elements of an array. You can do the same with an `ArrayList`. For example, earlier we mentioned that the following code could be used to add up the lengths of the `Strings` stored in an `ArrayList<String>` called `list`:

```
int sum = 0;
for (int i = 0; i < list.size(); i++) {
    String s = list.get(i);
    sum += s.length();
}
System.out.println("Total of lengths = " + sum);
```

We can simplify this code with a for-each loop. Remember that the syntax of this kind of loop is as follows:

```
for (<type> <name>: <structure>) {
    <statement>;
    <statement>;
    ...
    <statement>;
}
```

Thus, the preceding loop to add up the lengths of the `Strings` can be rewritten as follows:

```
int sum = 0;
for (String s: list) {
    sum += s.length();
}
System.out.println("Total of lengths = " + sum);
```

This loop is another way of saying, “For each `String s` contained in `list`”

Because the for-each loop has such a simple syntax, you should use it whenever you want to process each value stored in a list sequentially. You will find, however, that the for-each loop is not appropriate for more complex list problems. For example, there is no simple way to skip around in a list using a for-each loop. You must process the values in sequence from first to last. Also, you cannot modify the list while you are iterating over it.

Consider, for example, the following sample code:

```
// this doesn't work
for (String s: words) {
```

```
System.out.println(s);  
words.remove(0);  
}
```

This code prints a `String` from the list and then attempts to remove the value at the front of the list. When you execute this code, the program halts with a `ConcurrentModificationException`. Java is letting you know that you are not allowed to iterate over the list and to modify the list at the same time (concurrently). Because of this limitation, neither of the problems discussed in the previous section could be solved using a for-each loop.

Chapter 7  also mentioned that starting with version 8 of Java, there is a different approach to manipulating structures like arrays and lists that allows you to write more concise code. The following two lines of code perform the same task as the two loop versions above.

```
int sum = list.stream().mapToInt(String::length).sum();  
System.out.println("Total of lengths = " + sum);
```

You can read more about this approach in **Chapter 19** .

Wrapper Classes



So far, all of the `ArrayList` examples we have studied have involved `ArrayLists` of `String` objects. What if you wanted to form a list of integers? Given that `ArrayList<E>` is a generic class, you'd think that Java would allow you to define an `ArrayList<int>`, but that is not the case. The `E` in `ArrayList<E>` can be filled in with any object or reference type (i.e., the name of a class). The primitive types (e.g., `int`, `double`, `char`, and `boolean`), cannot be used as type parameters for an `ArrayList`.

Instead, Java defines a series of *wrapper classes* that allow you to store primitive data as objects.

Wrapper Class

A class that “wraps” (stores) primitive data as an object.

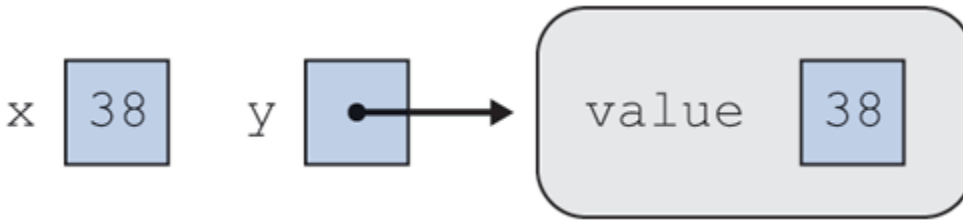
To understand the role of a wrapper class, think of a piece of candy packaged in a wrapper. Pieces of candy can be sticky and inconvenient to handle directly, so we put them inside wrappers that make handling them more convenient. When we want to eat the actual candy, we open up the wrapper to get the candy out. The Java wrapper classes fill a similar role.

For example, consider simple integers, which are of type `int`, a primitive type. Primitive types are not objects, so we can't use values of type `int` in an object context. To allow such use, we must wrap up each `int` into an object of type `Integer`. `Integer` objects are very simple. They have just one field: an `int` value. When we construct an `Integer`, we pass an `int` value to be wrapped; when we want to get the `int` back, we call a method called `intValue` that returns the `int`.

To understand the distinction between `int` and `Integer`, consider the following variable declarations:

```
int x = 38;  
Integer y = new Integer(38);
```

This code leads to the following situation in memory:



Primitive data is stored directly, so the variable `x` stores the actual value `38`. Objects, by contrast, are stored as references, so the variable `y` stores a reference to an object that contains `38`.

If we later want to get the `38` out of the object (to unwrap it and remove the candy inside), we call the method `intValue`:

```
int number = y.intValue();
```

The wrapper classes are of particular interest in this chapter because when you use an `ArrayList<E>`, the `E` needs to be a reference type. You can't make an `ArrayList<int>`, but you can form an `ArrayList<Integer>`. For example, you can write code like the following that enters several integer values into a list and adds them together:

```
ArrayList<Integer> list = new ArrayList<>();  
list.add(13);
```

```
list.add(47);  
list.add(15);  
list.add(9);  
int sum = 0;  
for (int n: list) {  
    sum += n;  
}  
System.out.println("list = " + list);  
System.out.println("sum = " + sum);
```

This code produces the following output:

```
list = [13, 47, 15, 9]  
sum = 84
```

The code takes advantage of a mechanism that Java provides for simplifying code which involves the use of wrapper classes. For example, Java will convert between `Integer` values and `int` values for you when your intent seems clear. Given the declaration of the variable `list` as an `ArrayList<Integer>`, Java would normally expect you to add values of type `Integer` to the list. But in the preceding code you were adding simple `int` values, as in:

```
list.add(13);
```

When it reaches this line of code, Java sees that you are adding an `int` to a structure that is expecting an `Integer`. Because Java understands the relationship between `int` and `Integer` (each `Integer` is simply an `int` wrapped up as an object), it will automatically convert the `int` value into a corresponding `Integer` object. This process is known as *boxing*.

Boxing

An automatic conversion from primitive data to a wrapped object of the appropriate type (e.g., an `int` boxed to form an `Integer`).

Similarly, you don't have to do anything special to unwrap an `Integer` to get the `int` inside. You could write code like the following:

```
int product = list.get(0) * list.get(1);
```

This code multiplies two values from the `ArrayList<Integer>` and stores the result in a variable of type `int`. The calls on `get` will return an `Integer` object, so normally these values would be incompatible. However, because Java understands the relationship between `int` and `Integer` it will unwrap the `Integer` objects for you and give you the `int` values stored inside. This process is known as *unboxing*.

Unboxing

An automatic conversion from a wrapped object to its corresponding primitive data (e.g., an `Integer` unboxed to yield an `int`).

Notice that you can write a for-each loop to use a variable of type `int` even though the `ArrayList` stores values of type `Integer`. Java will unbox the objects and perform the appropriate conversions for you.

Because Java has boxing and unboxing, the only place you generally need to use the wrapper class is when you describe a type like `ArrayList<Integer>`. You can't actually declare it to be of type `ArrayList<int>`, but you can manipulate it as if it is of type `ArrayList<int>`.

Table 10.3  lists the major primitive types and their corresponding wrapper classes.

Table 10.3 Common Wrapper Classes

10.2 The Comparable Interface

The method `Collections.sort` can be used to sort an `ArrayList`. It is part of the `java.util` package. The following short program demonstrates how to use `Collections.sort`:

```
1 // Constructs an ArrayList of Strings and sorts it.
2
3 import java.util.*;
4
5 public class SortExample {
6     public static void main(String[] args) {
7         ArrayList<String> words = new ArrayList<>();
8         words.add("four");
9         words.add("score");
10        words.add("and");
11        words.add("seven");
12        words.add("years");
13        words.add("ago");
14
15        // show list before and after sorting
16        System.out.println("before sort, words = " +
17        words);
17        Collections.sort(words);
```

```
18         System.out.println("after sort, words = " +  
words);  
19     }  
20 }
```

This program produces the following output:

```
before sort, words = [four, score, and, seven, years, ago]  
after sort, words = [ago, and, four, score, seven, years]
```

In [Chapter 13](#) , we will explore how this sorting method actually works. For now we are simply going to be clients of the method without worrying about how it works.

If you try to make a similar call to an `ArrayList<Point>`, you will find that the program does not compile. Why is it possible to sort a list of `String` objects but not a list of `Point` objects? The answer is that the `String` class implements the `Comparable` interface, while the `Point` class does not. In this section we will explore the details of the `Comparable` interface and explain how to write classes that implement it.

Controversy over Boxing and Unboxing

Not all software developers are happy with the decision to add boxing and unboxing to the Java language. The ability to manipulate an `ArrayList<Integer>` almost as if it were an `ArrayList<int>` can simplify code, and everyone agrees that simplification is good. The disagreement comes from the fact that it is *almost* like an `ArrayList<int>`. Some argue that “almost” isn’t good enough. Because it comes close, programmers are likely to use it and eventually come to count on it. That can prove disastrous when “almost” isn’t “always.”

As an analogy, suppose someone told you that you could use a device that is almost like a potholder to pick up hot objects. In most cases, it will protect your hand from heat. So you start using it, and while you might be nervous at first, you soon find that it seems to work just fine. And then one day you pick up a new object and you get burned. You can think of similar analogies with aircraft landing gear that almost works or vests that are almost bulletproof.

For a programming example, consider the following code:

```
int n = 420;

ArrayList<Integer> list = new ArrayList<>();

list.add(n);

list.add(n);

if (list.get(0) == list.get(1)) {

    System.out.println("equal");

} else {

    System.out.println("unequal");

}
```

It's difficult to know exactly what this code will do. If you assume that `ArrayList<Integer>` is “almost” like an `ArrayList<int>`, you'd probably think that the code would print the message that the two values are equal. In fact, there is no guarantee as to what it will do. In the current version of Java, it prints the message “unequal.”

Remember that testing for object equality is not as simple as testing for equality of primitive data. Two `String`s might store the same text but might not be the same object, which is why we call the `equals` method to compare `String`s. The same principle applies here: The two `list` elements might store the same `int` but might not be the same object. The code prints “unequal” in the current release of Java because the program creates two different `Integer` objects that each store the value

420. However, to add to the confusion, if we change the value from 420 to 42, the program will print that the two values are equal.

The Java Language Specification guarantees that this code will work for any value of `n` between -128 and 127, but it provides no guarantee as to how the code will behave for other values of `n`. For those other values, it could print either message, and this might change from one implementation of Java to another. It might be that in the next version of Java released, the code will print “equal” for 420 but not for a value like 420000.

Some people have argued that because boxing and unboxing cover up what is happening underneath, it is better not to use them at all. Boxing and unboxing don’t necessarily simplify anything if they work only “sometimes,” because you have to be able to understand the cases in which they don’t work.

Natural Ordering and `compareTo`

We are all familiar with many kinds of data that can be sorted. For example, we are used to putting numbers in order from lowest to highest or alphabetizing lists of names. We describe types that can be sorted as having a *natural ordering* of values. To have such an

ordering of values, a type needs to have a well-defined *comparison function* that indicates the relationship between any pair of values.

Comparison Function

A well-defined procedure for deciding, given a pair of values, the relative order of the two values (less than, equal to, or greater than).

Natural Ordering

The order imposed on a type by its comparison function.

Not all types have natural orderings because not all types have comparison functions. For example, in this chapter we have been exploring how to construct a variety of `ArrayList` objects. How would you compare two `ArrayList` objects to determine whether one is less than another? What would it mean for one `ArrayList` to be less than another? You might decide to use the lengths of the lists to

determine which one is less, but what would you do with two `ArrayList` objects of equal length that store different values? You wouldn't want to describe them as "equal." There is no agreed-upon way of ordering `ArrayList`s, and therefore there is no comparison function for this type. As a result, we say that the `ArrayList` type does not have a natural ordering.

Java has a convention for indicating the natural ordering of a type. Any type that has such an ordering should implement the `Comparable` interface:

```
public interface Comparable<T> {  
    public int compareTo(T other);  
}
```

This interface provides a second example of a generic type in Java. In the case of `ArrayList`, Java uses the letter "E," which is short for "Element." In the case of `Comparable`, Java uses the letter "T," which is short for "Type."

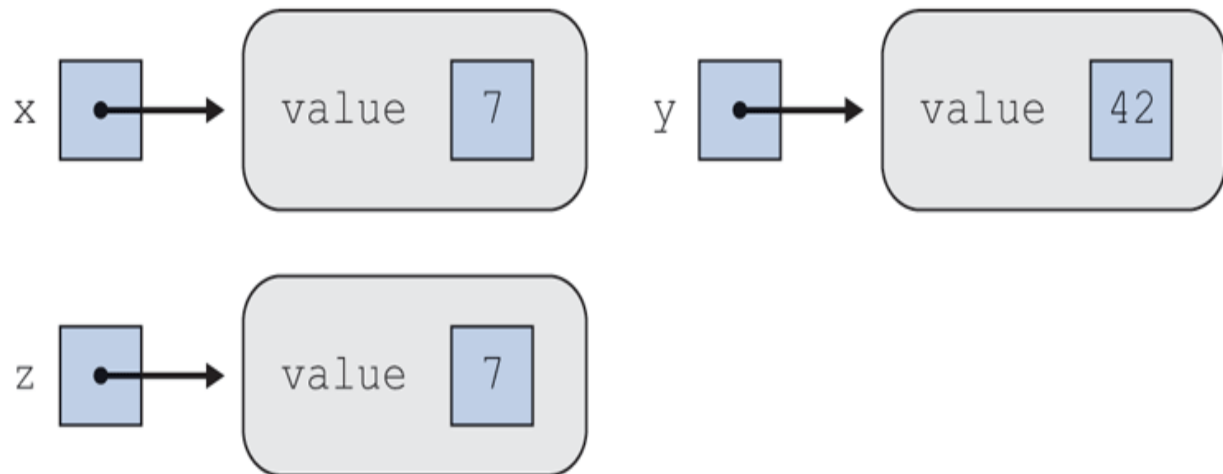
The `compareTo` method is the comparison function for the type. A boolean return type can't be used because there are three possible answers: less than, equal to, or greater than. The convention for `compareTo` is that an object should return one of the following results:

- A negative number to indicate a less-than relationship
- 0 to indicate equality
- A positive number to indicate a greater-than relationship

Let's look at a few examples to help you understand this concept. We have seen that Java has `Integer` objects that serve as wrappers for individual `int` values. We know how to compare `int` values to determine their relative order, so it is not surprising that the `Integer` class implements the `Comparable` interface. Consider the following code:

```
Integer x = 7;  
Integer y = 42;  
Integer z = 7;  
System.out.println(x.compareTo(y));  
System.out.println(x.compareTo(z));  
System.out.println(y.compareTo(x));
```

This code begins by constructing three `Integer` objects:



Then it includes a series of `println` statements that report the results of various pairwise comparisons. In the first `println` statement, we compare `x` to `y`, which involves comparing the `int` value `7` to the `int` value `42`. This pair has a less-than relationship because `x` is less than `y`, so the method call returns a negative integer. In the second `println` statement, we compare `x` to `z`, which involves comparing one occurrence of the `int` value `7` with another occurrence of the `int` value `7`. This second pair has an equality relationship because `x` equals `z`, so the method call returns `0`. In the final `println` statement, we compare `y` to `x`, which involves comparing the `int` value `42` to the `int` value `7`. This final pair has a greater-than relationship because `y` is greater than `x`, so the method call returns a positive integer.

Here is the actual output of the code:

```
-1  
0  
1
```

The values `-1`, `0`, and `1` are the standard values returned, but the `compareTo` method is not required to return these specific values. For example, consider a similar piece of code that compares `String` values:

```
String x = "hello";  
String y = "world";  
String z = "hello";  
System.out.println(x.compareTo(y));  
System.out.println(x.compareTo(z));  
System.out.println(y.compareTo(x));
```

The `compareTo` method of the `String` class compares strings alphabetically, so there are similar relationships in this code: `x` is less than `y` because in an alphabetical list `"hello"` comes before `"world"`, `x` is equal to `z` because the two occurrences of `"hello"` are equal, and `y` is greater than `x` because in an alphabetical list

`"world"` comes after `"hello"`. But the output produced is slightly different from that produced by the `Integer` example:

```
-15  
0  
15
```

Instead of `-1` and `1`, we get `-15` and `15`. You don't really need to know where these numbers come from—the only important fact is whether they are negative or positive—but for those of you who are curious, the `-15` and `15` represent the distance between the positions of the characters `'h'` and `'w'` in type `char`. `'w'` appears 15 positions later than `'h'`.

So while the values `-1` and `1` are often returned by a comparison function, that won't always be the case. The important thing to remember is that “less-than” relationships are indicated by a negative number and “greater-than” relationships are indicated by a positive number.

Also keep in mind that the relationship operators that we've been using since [Chapter 4](#)  have a different syntax. For example, you've seen that if two variables `x` and `y` are of type `int` or `double`, you can compare them by using operators like `<` and `>`:

```
int x = 7;
int y = 42;
if (x < y) {
    System.out.println("x less than y");
}
```

Even though the `String` class implements the `Comparable` interface, you can't use the relational operators to compare `Strings`. The following code will not compile:

```
// illegal--can't compare objects this way
String s1 = "hello";
String s2 = "world";
if (s1 < s2) {
    System.out.println("s1 less than s2");
}
```

Instead, call the `compareTo` method, as in:

```
String s1 = "hello";
String s2 = "world";
if (s1.compareTo(s2) < 0) {
```

```
System.out.println("s1 less than s2");  
}
```

You can use a relational operator in this context because the `compareTo` method returns an `int`. Notice that the specific value of `-1` isn't used for `compareTo` because you are only guaranteed to get a negative value for a less-than relationship. [Table 10.4](#)  summarizes the standard way to compare objects that implement the `Comparable` interface.

Table 10.4 Comparing Values Summary

Implementing the `Comparable` Interface



Many of the standard Java classes, such as `String`, implement the `Comparable` interface. You can have your own classes implement the

interface as well. Implementing the `Comparable` interface will open up a wealth of off-the-shelf programming solutions that are included in the Java class libraries. For example, there are built-in methods for sorting lists and for speeding up searches. Many of these features will be discussed in the next chapter.

As a fairly simple example, let's explore a class that can be used to keep track of a calendar date. The idea is to keep track of a particular month and day, but not the year. For example, the United States celebrates its independence on July 4 each year. Similarly, an organization might want a list of its employees' birthdays that doesn't indicate how old they are.

DID YOU KNOW?

Why Not `-1`, `0`, and `1` for `compareTo`?

As we discussed earlier, some types like `Integer` return the values `-1`, `0`, and `1` when you call `compareTo`. These are the *canonical* values for `compareTo` to return, because they correspond to a function in mathematics known as the *signum* function (sometimes abbreviated “sgn”). However, other types like `String` do not return the standard values. You might

wonder why Java doesn't require that all classes return `-1`, `0`, and `1` when you call `compareTo`.

One answer to this question is that Java doesn't have a convenient ternary type. For any binary decision, we can use `boolean` as the return type. But what type do we use if we want to return exactly one of three different values? There is no predefined type that has just three values, so it's more honest in a sense to use the rule that any negative number will do and any positive number will do. Suppose that Java said that `compareTo` should return only `-1`, `0`, and `1`. What should happen when someone writes a `compareTo` that returns something else? Ideally, any code calling that particular `compareTo` would throw an exception when it gets an illegal return value, but that would require programmers to write a lot of error-checking code. By saying that all negatives will be interpreted one way, all positives will be interpreted a second way, and `0` will be interpreted a third way, Java provides a complete definition for all values of type `int`, which makes it easier for programmers to work with the `compareTo` method.

A second reason for having `compareTo` behave this way is that it then is easy to express many comparison tasks directly. It is often convenient to express the comparison as a difference between two values. This pattern occurs in many places. For example, the `String` class uses *lexicographic* order (also called "dictionary" or "alphabetic" order). To determine the relationship between two `Strings`, you scan through them

until you find the first pair of letters that differ. For example, if you were comparing `"nattering"` and `"nabobs"`, you'd find that the first pair of characters that differ is the third pair (`"nat..."` versus `"nab..."`). You would then return the difference between the character values (`'t' - 'b'`). If you don't find such a pair, then you return the difference between the lengths. For example, `"nattering"` is considered greater than `"nat"` on the basis of length.

The `compareTo` behavior for the `String` class can be described with the following pseudocode:

```
search for a pair of characters in corresponding
positions that differ.
if (such a pair exists) {
    return the difference between the two characters.
} else {
    return the difference between the two lengths.
}
```

Notice that this approach returns `0` in just the right case, when there are no character pairs that differ and when the strings have the same length. Having the flexibility to return any negative integer for “less than” and any positive integer for “greater than” makes it easier to implement this approach.

The final reason not to specify the return values for `compareTo` is efficiency. By having a less strict rule, Java allows programmers to write faster `compareTo` methods. The `compareTo` method in the `String` class is one of the most frequently called methods. All sorts of data comparisons are built on `String` comparisons, and performing a task like sorting thousands of records will lead to thousands of calls on the `String` class's `compareTo` method. As a result, it's important for the method to run quickly. We wouldn't want to unnecessarily complicate the code by requiring that it always return `-1`, `0`, or `1`.

We can implement this class of dates with two fields to store the month and day:

```
public class CalendarDate {  
    private int month;  
    private int day;  
  
    public CalendarDate(int month, int day) {  
        this.month = month;  
        this.day = day;  
    }  
    // other methods  
}
```

Remember that to implement an interface, you include an extra notation in the class header. Implementing the `Comparable` interface is a little more challenging because it is a generic interface (`Comparable<T>`). We can't simply write code like the following:

```
// not correct
public class CalendarDate implements Comparable {
    ...
}
```

We have to replace the `<T>` in `Comparable<T>`. Whenever you implement `Comparable`, you compare pairs of values from the same class. So a class called `CalendarDate` should implement `Comparable<CalendarDate>`. If you look at the header for the `Integer` class, you will find that it implements `Comparable<Integer>`. Likewise, the `String` class implements `Comparable<String>`. So we need to change the header to the following one:

```
public class CalendarDate implements Comparable<CalendarDate> {
    ...
}
```

Of course, claiming to implement the interface is not enough. We also have to include appropriate methods. In this case, the `Comparable` interface includes just a single method:

```
public interface Comparable<T> {  
    public int compareTo(T other);  
}
```

Because we are using `CalendarDate` in place of `T`, we need to write a `compareTo` method that takes a parameter of type `CalendarDate`:

```
public int compareTo(CalendarDate other) {  
    ...  
}
```

Now we have to figure out how to compare two dates. Each `CalendarDate` object will contain fields that store the month and day. With calendars, the month takes precedence over the day. If we want to compare January 31 (1/31) with April 5 (4/5), we don't care that 5 comes before 31; we care instead that January comes before April. So, as a first attempt, consider the following method that compares only the months:

```
// compares only the months
public int compareTo(CalendarDate other) {
    if (month < other.month) {
        return -1;
    } else if (month == other.month) {
        return 0;
    } else { // month > other.month
        return 1;
    }
}
```

We need to consider more than just the month, but before we do that, we can improve on what we have here. This code uses a nested `if/else` construct to return the standard values of `-1`, `0`, and `1`, but a simpler option is available. We can simply return the difference between `month` and `other.month`, because it will be negative when `month` is less than `other.month`, it will be `0` when they are equal, and it will be positive when `month` is greater than `other.month`. So, we can simplify the code as follows:

```
// still uses only the month, but more compact
public int compareTo(CalendarDate other) {
    return month - other.month;
}
```

It is a good idea to keep things simple when you can, so this version is preferable to the original one. It returns slightly different values than the earlier version, but it satisfies the contract of the `Comparable` interface just as well. However, the code still has a problem.

Consider, for example, a comparison of April 1 (4/1) and April 5 (4/5). The current version of `compareTo` would subtract the months and return a value of `0`, indicating that these two dates are equal. However, the dates aren't equal: April 1 comes before April 5.

The day of the month becomes important only when the months are equal. If the months differ, we can use the months to determine order. Otherwise (when the months are equal), we must use the day of the month to determine order. This common ordering principle is present in many tasks. We can implement it as follows:

```
public int compareTo(CalendarDate other) {  
    if (month != other.month) {  
        return month - other.month;  
    } else {  
        return day - other.day;  
    }  
}
```

It is still possible for this code to return `0`. Suppose that we have two `CalendarDate` objects that both store the date April 5 (4/5). The months are equal, so the program returns the difference between the days. That difference is 0, so the program returns `0`, which correctly indicates that the two dates are equal.

Here is a complete `CalendarDate` class with the `compareTo` method, two accessor methods, and a `toString` method:

```
1  // The CalendarDate class stores information about a single
2  // calendar date (month and day but no year).
3
4  public class CalendarDate implements
Comparable<CalendarDate> {
5      private int month;
6      private int day;
7
8      public CalendarDate(int month, int day) {
9          this.month = month;
10         this.day = day;
11     }
12
13     // Compares this calendar date to another date.
14     // Dates are compared by month and then by day.
15     public int compareTo(CalendarDate other) {
16         if (month != other.month) {
```

```

17         return month - other.month;
18     } else {
19         return day - other.day;
20     }
21 }
22
23 public int getMonth() {
24     return month;
25 }
26
27 public int getDay() {
28     return day;
29 }
30
31 public String toString() {
32     return month + "/" + day;
33 }
34 }

```

One of the major benefits of implementing the `Comparable` interface is that it gives you access to built-in utilities like `Collections.sort`. As we mentioned previously, you can use `Collections.sort` to sort an `ArrayList<String>` but not to sort an `ArrayList<Point>`, because the `Point` class does not implement `Comparable`. The `CalendarDate` class implements the `Comparable` interface, so, as the following short program demonstrates, we can use `Collections.sort` for an `ArrayList<CalendarDate>`:

```

1  // Short program that creates a list of the birthdays of
the
2  // first 5 U.S. Presidents and that puts them into sorted
order.
3
4  import java.util.*;
5
6  public class CalendarDateTest {
7      public static void main(String[] args) {
8          ArrayList<CalendarDate> dates = new ArrayList<>();
9          dates.add(new CalendarDate(2, 22));    // Washington
10         dates.add(new CalendarDate(10, 30));    // Adams
11         dates.add(new CalendarDate(4, 13));     // Jefferson
12         dates.add(new CalendarDate(3, 16));     // Madison
13         dates.add(new CalendarDate(4, 28));     // Monroe
14
15         System.out.println("birthdays = " + dates);
16         Collections.sort(dates);
17         System.out.println("birthdays = " + dates);
18     }
19 }

```

This program produces the following output:

```
birthdays = [2/22, 10/30, 4/13, 3/16, 4/28]  
birthdays = [2/22, 3/16, 4/13, 4/28, 10/30]
```

Notice that the dates appear in increasing calendar order after the call on `Collections.sort`.

10.3 Case Study: Ranked Choice Voting

In this section, we will use `ArrayList`s to solve a complex problem. We will develop a program that implements what is known as *ranked choice voting*. Ranked choice voting has been getting more attention in the United States in recent years with cities like San Francisco and Minneapolis using it along with the state of Maine.

Ranked choice voting (also called “instant runoff”) allows voters to specify what order they prefer for various candidates. Suppose that you have eight voters and four candidates. To keep things simple, we will refer to candidates using the letters A, B, C, and D. Below is a list of voting preferences. Each line has a ranked list of the four candidates and represents a single voter’s preferences.

A	D	C	B
A	C	B	D
B	D	C	A
C	D	B	A
C	B	A	D
A	C	D	B

D	C	B	A
B	C	A	D

The usual voting procedure is to tally the first choice of each voter, which means looking at the votes in the first column. If you tally those votes, nobody ends up with a clear majority. Candidate A has three votes or 37.5% of the eight total votes. In ranked choice voting when no candidate has a majority, we eliminate the candidate with the lowest vote total, which in this case is candidate D. That leaves us with these preferences:

A	C	B
A	C	B
B	C	A
C	B	A
C	B	A
A	C	B
C	B	A
B	C	A

There is still no candidate with a majority, so we eliminate the one with the low vote total, which is candidate B leaving us with:

A	C
---	---

A	C
C	A
C	A
C	A
A	C
C	A
C	A

Now we see that candidate C has five votes and wins the election. Notice that in the first round of voting, candidate C was tied for second place. But more voters liked candidate C as a substitute if they couldn't get the candidates they preferred more.

We are going to develop a program that performs the ranked choice voting algorithm assuming we have complete preferences from each voter. As with other case studies, we will develop it in stages:

1. We will develop a class called `Ballot` to store information about one voter's preferences.
2. We will write a program that reads an input file of voter preferences and that tallies the results for each voter's first choice pick.
3. We will introduce code to repeatedly eliminate the candidate with the lowest vote total until some candidate gets a majority or we end up with a tie with two candidates each with the same vote total.

Ballot Class

In solving this problem, you can identify two kinds of data: the data for a single voter which we are calling a ballot, and the list of ballots. It makes sense to introduce a class for storing the information for a single ballot.

```
// A class for storing one voter's preferences.  
public class Ballot {  
    ...  
}
```

Assume that as with the lists shown previously, each voter's preferences will be listed as a single line of data with the candidates listed from most desirable to least desirable. One common way to store a file with text like this is to separate the different names with a tab character. Spreadsheet programs have an option to save data as a tab-delimited file that would have this structure. You will see that it is fairly easy to break up such a string into an array of strings. It makes sense, then, to assume the constructor will take an array of strings as a parameter:

```
public Ballot(String[] names) {
```

```
    ...  
}
```

You need to store this sequence of names. You could use an array, but you know that the algorithm involves eliminating candidates who get lower vote totals. It would be better to store the names in an

`ArrayList` that can be stored as a field of the class:

```
private ArrayList<String> preferences;
```

To complete the constructor, you need to construct this list and add each name to your list. This is a nice place to use a for-each loop:

```
// constructs a ballot with the given candidate names in order  
public Ballot(String[] names) {  
    preferences = new ArrayList<>();  
    for (String next : names) {  
        preferences.add(next);  
    }  
}
```

The ballot includes all of the candidates, but the code that processes a ballot needs to know which candidate is the current top choice.

That will always be the first element of your list, the one located at index 0. You can therefore provide a method for requesting the current choice of candidate by looking at the first element of the list:

```
// returns the current first choice for this ballot  
public String getCandidate() {  
    return preferences.get(0);  
}
```

As noted earlier, this approach will work only if each ballot contains information for all candidates. If you don't have complete information, then eventually the list of candidates can become empty. This case study assumes complete information, but you could extend the code to deal with incomplete information (e.g., when a voter doesn't want to include some candidate).

Another important behavior you have to implement is to eliminate a candidate from consideration. The `ArrayList` comes in handy for this because you can simply call its `remove` method to remove a value:

```
// eliminates the given candidate from consideration  
public void eliminate(String name) {  
    preferences.remove(name);  
}
```

You need one final behavior for this class. To make it easier to count votes, it would be helpful if a list of ballots could be sorted by candidate name. That way all of the votes for a given candidate will be grouped together. That means that you need to have the `Ballot` class implement the `Comparable` interface. The idea is to make it easier to count votes, so you want to use the candidate names to put them in order. The names are stored as strings and the `String` class implements `Comparable`. That means you can use the `compareTo` method of the individual strings that represent the candidate names to perform this comparison:

```
// compares ballots alphabetically by first choice candidate
public int compareTo(Ballot other) {
    return getCandidate().compareTo(other.getCandidate());
}
```

Putting all of these pieces together, you end up with the following complete class:

```
1 // A class for storing one voter's preferences.
2 import java.util.*;
3
4 public class Ballot implements Comparable<Ballot> {
```

```
5     private ArrayList<String> preferences;
6
7     // constructs a ballot with candidate names in order
8     public Ballot(String[] names) {
9         preferences = new ArrayList<>();
10        for (String next: names) {
11            preferences.add(next);
12        }
13    }
14
15    // returns the current first choice for this ballot
16    public String getCandidate() {
17        return preferences.get(0);
18    }
19
20    // eliminates the given candidate from consideration
21    public void eliminate(String name) {
22        preferences.remove(name);
23    }
24
25    // compares this ballot to another in order
26    // alphabetically by first choice candidate
27    public int compareTo(Ballot other) {
28        return
29        getCandidate().compareTo(other.getCandidate());
29    }
30 }
```

Counting Votes

Now that we have code to represent a ballot, we can turn to writing the overall program in a separate file called `TallyVotes.java`.

Assume that votes are stored in a file called `vote.txt`. The first version of your program is supposed to read the information from this file to create an `ArrayList` of `Ballot` objects. You can then sort the list by candidate name and count the votes. As with the calendar sample program, you can call `Collections.sort` to sort the list. The reading and vote counting operations deserve to be written as methods. That leads to the following `main` method:

```
public class TallyVotes {  
    public static void main(String[] args)  
        throws FileNotFoundException {  
        Scanner input = new Scanner(new File("vote.txt"));  
        ArrayList<Ballot> ballots = readFile(input);  
        Collections.sort(ballots);  
        oneRound(ballots);  
    }  
    ...  
}
```

The `readFile` method involves reading the input file line by line and adding `Ballot` objects to an `ArrayList`. The `Ballot` constructor

expects an array of strings to specify the candidate choice. Recall that the plan is to have each input line list the candidates with tabs between candidate names. The `String` class has a method called `split` that provides a convenient way to break up this string into the individual names. It takes a parameter that specifies what separator or delimiter to use. The reading method, then, can be written as follows:

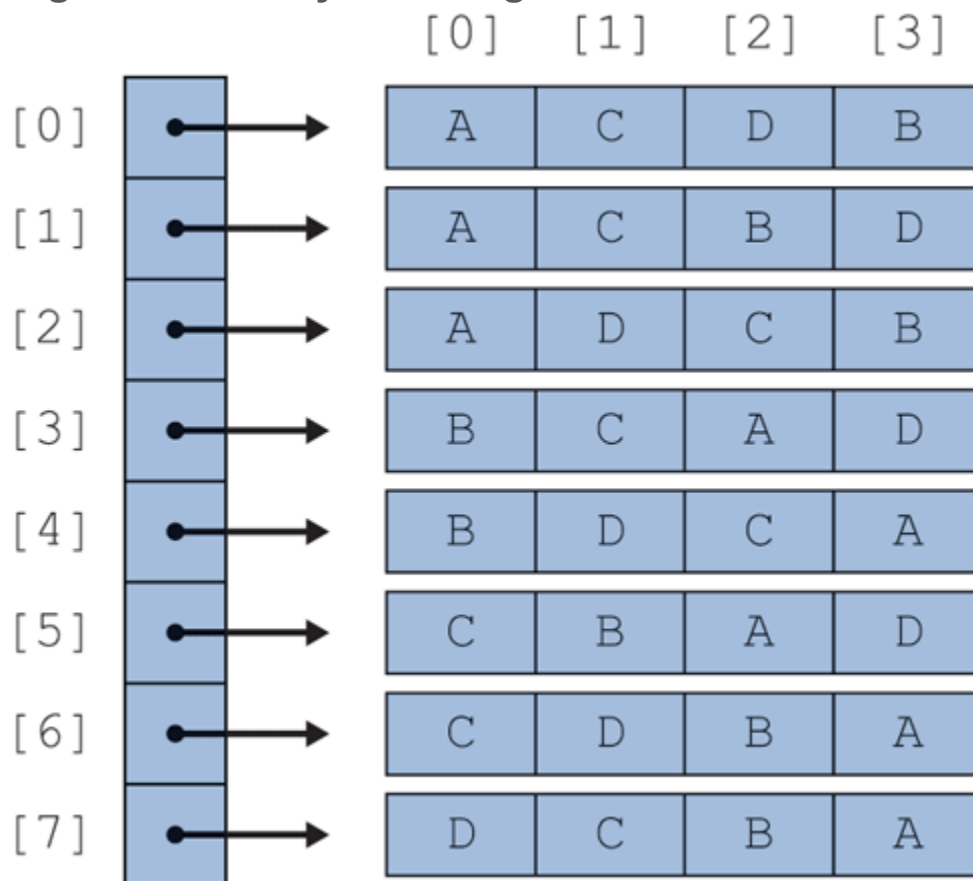
```
// Reads a data file of voter preferences, returning a list
// of the resulting ballots.
public static ArrayList<Ballot> readFile(Scanner input) {
    ArrayList<Ballot> result = new ArrayList<>();
    while (input.hasNextLine()) {
        String text = input.nextLine();
        result.add(new Ballot(text.split("\t")));
    }
    return result;
}
```

The `oneRound` method takes a bit more effort to complete. The `main` method calls `Collections.sort` to put the ballots in order by candidate name. That means that all of the votes for a particular candidate will be grouped together. If you use simple names like A, B, C, and D for candidates, then all of the votes for candidate A will come first followed by all of the votes for candidate B, and so on.

This problem involves two levels of structure that can be useful to separate into different methods. You need an outer loop that processes the first candidate, then the second candidate, and so on. For each candidate, you need to count how many votes that candidate got.

If you use the sample set of eight votes mentioned in the beginning of the section and you store them in an `ArrayList` ordered by the name of the candidate, you end up with the sequence shown in [Figure 10.4](#).

Figure 10.4 `ArrayList` of eight ballots



The list elements 0 through 2 represent votes for candidate A, which means that A got 3 votes. The list elements 3 through 4 represent votes for candidate B, which is two votes. To process this list candidate by candidate, you can keep track of a current index and then compute the number of votes for the next candidate. This vote count can then be added to the index to position to the beginning of the votes for the next candidate. For example, you can start the index at 0 for the first candidate and given that the first candidate got 3 votes, you know to increase the index to 3 to process votes for the second candidate.

In counting the votes, you need access to the list of ballots and you also need to know which index to start at. It isn't a bad idea to also provide the name of the candidate. The following code implements this strategy:

```
// Performs one round of ranked choice voting.
public static void oneRound(ArrayList<Ballot> ballots) {
    int index = 0;
    while (index < ballots.size()) {
        String next = ballots.get(index).getCandidate();
        int count = processVotes(next, index, ballots);
        index += count;
    }
}
```

The second level of structure is to implement the `processVotes` method that counts the votes for a single candidate. Remember that the method is passed the starting index, candidate name, and list of ballots as parameters:

```
// Counts and reports the votes for the next candidate
// starting at the given index in the ballots list.
public static int processVotes(String name, int index,
                               ArrayList<Ballot> ballots) {
    ...
}
```

It needs to have a loop that increments the index while the next name in the list matches the given candidate, counting as it goes. The following first attempt is not quite right:

```
int count = 0;
while (ballots.get(index).getCandidate().equals(name)) {
    index++;
    count++;
}
```

To see the bug in the preceding code, ask yourself, what happens when you reach the last candidate? You can't count on finding

another element of the list with a different name. That means you need an additional test to make sure the index hasn't become too high. You have to put the new test first so that you avoid getting an

`IndexOutOfBoundsException`:

```
int count = 0;
while (index < ballots.size() &&
      ballots.get(index).getCandidate().equals(name)) {
    index++;
    count++;
}
```

Once you know the count for this candidate, you can compute and report their percentage of the vote. Since we want to round the results, this is a good place to use a `printf` statement:

```
double percent = 100.0 * count / ballots.size();
System.out.printf("%d votes for %s (%4.1f%%)\n", count,
                  name, percent);
```

The final step for the method is to return the count. Putting these pieces together, you get the following complete program:

```

1  // This program reads a data file and counts candidates'
votes.
2
3  import java.util.*;
4  import java.io.*;
5
6  public class TallyVotes1 {
7      public static void main(String[] args)
8          throws FileNotFoundException {
9          Scanner input = new Scanner(new File("vote.txt"));
10         ArrayList<Ballot> ballots = readFile(input);
11         Collections.sort(ballots);
12         oneRound(ballots);
13     }
14
15     // Reads a data file of voter preferences, returning a
list
16     // of the resulting ballots.  Candidate names are listed
in
17     // order of preference with tabs separating choices.
18     public static ArrayList<Ballot> readFile(Scanner input)
{
19         ArrayList<Ballot> result = new ArrayList<>();
20         while (input.hasNextLine()) {
21             String text = input.nextLine();
22             result.add(new Ballot(text.split("\t")));
23         }

```

```

24         return result;
25     }
26
27     // Counts and reports votes for each candidate. Assumes
the
28     // list is in order by candidate name.
29     public static void oneRound(ArrayList<Ballot> ballots)
{
30         int index = 0;
31         while (index < ballots.size()) {
32             String next =
ballots.get(index).getCandidate();
33             int count = processVotes(next, index, ballots);
34             index += count;
35         }
36     }
37
38     // Counts and reports the votes for the next candidate
39     // starting at the given index in the ballots list.
40     public static int processVotes(String name, int index,
41                                     ArrayList<Ballot>
ballots) {
42         int count = 0;
43         while (index < ballots.size() &&
44
ballots.get(index).getCandidate().equals(name)) {
45             index++;
46             count++;

```

```

47         }
48         double percent = 100.0 * count / ballots.size();
49         System.out.printf("%d votes for %s (%4.1f%%)\n",
count,
50         name, percent);
51         return count;
52     }
53 }

```

When you run the program using the short input file of eight votes, you get the following output:

```

3 votes for A (37.5%)
2 votes for B (25.0%)
2 votes for C (25.0%)
1 votes for D (12.5%)

```

The program does not yet eliminate candidates to find the overall election winner, but we'll add that functionality in the next section.

Multiple Rounds

The current version of the program counts the votes for each candidate and returns those counts to the method called `oneRound`.

The remainder of the program involves multiple rounds where candidates are eliminated until a winner is found. You can modify the `oneRound` method to return a `boolean` value indicating whether or not the voting has ended. If you add some code for reporting the round number and include a line of dashes to separate the different rounds of output, you end up with the following modifications to our `TallyVotes` program's `main` method. Notice that we have to sort the ballots after each round to order them by each voter's next preferred candidate:

```
public static void main(String[] args)
    throws FileNotFoundException {
    Scanner input = new Scanner(new File("vote.txt"));
    ArrayList<Ballot> ballots = readFile(input);
    int round = 1;
    boolean done = false;
    while (!done) {
        System.out.println("Round #" + round);
        Collections.sort(ballots);
        done = oneRound(ballots);
        System.out.println("-----");
        round++;
    }
}
```

The header of the `oneRound` method will have to be changed to reflect the fact that it returns a `boolean` result.

```
public static boolean oneRound(ArrayList<Ballot> ballots) {  
    ...  
}
```

The method has a loop that processes one candidate at a time. To determine whether there is a winner, you need to know which candidate got the most votes. If there is no winner, you have to eliminate the candidate with the lowest vote total, which means you would need to also know which candidate got the lowest number of votes. These are classic max/min computations. You need to know both the candidate names and their vote totals, so there are a lot of variables involved, but otherwise this is a straightforward task of including the standard max/min code to the existing loop. You can initialize the minimum to one more than the total number of ballots and initialize the maximum to 0, and you can initialize the candidate names to `null`, which leads to the following code:

```
// oneRound implementation (partial)  
String top = null;  
String bottom = null;
```

```

int topCount = 0;
int bottomCount = ballots.size() + 1;
int index = 0;
while (index < ballots.size()) {
    String next = ballots.get(index).getCandidate();
    int count = processVotes(next, index, ballots);
    if (count > topCount) {
        topCount = count;
        top = next;
    }
    if (count < bottomCount) {
        bottomCount = count;
        bottom = next;
    }
    index += count;
}

```

There are three possible outcomes. If the count for the top candidate is more than 50%, then that candidate has won. Otherwise you normally would eliminate the candidate with the lowest vote total. But there is one special case that is important to consider. When you get down to just two candidates, there might be a tie where each of the remaining candidates earns exactly 50% of the vote. In that case, there is no winner and you would want to stop the algorithm from continuing. The task of eliminating a candidate deserves to be its own method. The rest of the code can be written as a nested `if/else` that distinguishes these three cases, producing an

appropriate message and returning either `true` or `false` depending upon whether the voting has ended. That leads to the following code:

```
// determining a winner
if (topCount == bottomCount) {
    System.out.println("Election has no winner");
    return true;
} else if (topCount > ballots.size() / 2.0) {
    System.out.println("Winner is " + top);
    return true;
} else {
    System.out.println("no winner, eliminating " + bottom);
    eliminate(bottom, ballots);
    return false;
}
```

The task of eliminating a candidate simply involves calling the `Ballot` object's `eliminate` method for each of the ballots in the list. This is a good place to use a for-each loop:

```
// Eliminates the given candidate from all ballots.
public static void eliminate(String candidate,
                             ArrayList<Ballot> ballots) {
    for (Ballot b: ballots) {
```

```

        b.eliminate(candidate);
    }
}

```

Putting these pieces together, you get the following complete program:

```

1  // Program to perform ranked choice voting algorithm
2  // using a data file of voting preferences.
3
4  import java.util.*;
5  import java.io.*;
6
7  public class TallyVotes2 {
8      public static void main(String[] args)
9          throws FileNotFoundException {
10         Scanner input = new Scanner(new File("vote.txt"));
11         ArrayList<Ballot> ballots = readFile(input);
12         int round = 1;
13         boolean done = false;
14         while (!done) {
15             System.out.println("Round #" + round);
16             Collections.sort(ballots);
17             done = oneRound(ballots);
18             System.out.println("-----
--");

```

```
19         round++;
20     }
21 }
22
23 // Reads a data file of voter preferences, returning a
list
24 // of the resulting ballots.  Candidate names are listed
in
25 // order of preference with tabs separating choices.
26 public static ArrayList<Ballot> readFile(Scanner input)
{
27     ArrayList<Ballot> result = new ArrayList<>();
28     while (input.hasNextLine()) {
29         String text = input.nextLine();
30         result.add(new Ballot(text.split("\t")));
31     }
32     return result;
33 }
34
35 // Performs one round of ranked choice voting.  The
candidate
36 // with the lowest vote total is eliminated until some
37 // candidate gets a majority or until we reach a tie
between
38 // only two candidates.  Assumes the list is in order
by
39 // candidate name.
40 public static boolean oneRound(ArrayList<Ballot>
```

```

ballots) {
41         String top = null;
42         String bottom = null;
43         int topCount = 0;
44         int bottomCount = ballots.size() + 1;
45         int index = 0;
46         while (index < ballots.size()) {
47             String next =
ballots.get(index).getCandidate();
48             int count = processVotes(next, index, ballots);
49             if (count > topCount) {
50                 topCount = count;
51                 top = next;
52             }
53             if (count < bottomCount) {
54                 bottomCount = count;
55                 bottom = next;
56             }
57             index += count;
58         }
59         if (topCount == bottomCount) {
60             System.out.println("Election has no winner");
61             return true;
62         } else if (topCount > ballots.size() / 2.0) {
63             System.out.println("Winner is " + top);
64             return true;
65         } else {
66             System.out.println("no winner, eliminating " +

```

[illegible]

```

{
91     for (Ballot b: ballots) {
92         b.eliminate(candidate);
93     }
94 }
95 }

```

Given the previous short input file of eight votes, it produces the following output:

```

Round #1
3 votes for A (37.5%)
2 votes for B (25.0%)
2 votes for C (25.0%)
1 votes for D (12.5%)
no winner, eliminating D
-----

Round #2
3 votes for A (37.5%)
2 votes for B (25.0%)
3 votes for C (37.5%)
no winner, eliminating B
-----

Round #3
3 votes for A (37.5%)

```

5 votes for C (62.5%)

Winner is C

Chapter Summary

The `ArrayList` class in Java's `java.util` package represents a growable list of objects implemented using an array. You can use an `ArrayList` to store objects in sequential order. Each element has a zero-based index.

`ArrayList` is a generic class. A generic class accepts a data type as a parameter when it is created, like `ArrayList<String>`.

An `ArrayList` maintains its own size for you; elements can be added to or removed from any position up to the size of the list. Other `ArrayList` operations include `get`, `set`, `clear`, and `toString`.

`ArrayLists` can be searched using methods named `contains`, `indexOf`, and `lastIndexOf`.

Java's for-each loop can be used to examine each element of an `ArrayList`. The list cannot be modified during the execution of the for-each loop.

When you are storing primitive values such as `ints` or `doubles` into an `ArrayList`, you must declare the list with special wrapper types such as `Integer` and `Double`.

The `Comparable` interface defines a natural ordering for the objects of a class. Objects that implement `Comparable` can be placed into an `ArrayList` and sorted. Many common types (such as `String` and `Integer`) implement `Comparable`.

You can implement the `Comparable` interface in your own classes by writing a method `compareTo`.

Self-Check Problems

Section 10.1: ArrayLists

1. What is an `ArrayList`? In what cases should you use an `ArrayList` rather than an array?
2. Which of the following is the correct syntax to construct an `ArrayList` to store integers?
 - a. `ArrayList list = new ArrayList();`
 - b. `ArrayList[int] list = new ArrayList[int]();`
 - c. `ArrayList list<> = new ArrayList<integer>();`
 - d. `ArrayList<Integer> list = new ArrayList();`
 - e. `ArrayList<Integer> list = new ArrayList<>();`
3. The next five questions refer to the following `String` elements:

```
["It", "was", "a", "stormy", "night"]
```

Write the code to declare an `ArrayList` containing these elements. What is the size of the list? What is its type?

4. Write code to insert two additional elements, `"dark"` and `"and"`, at the proper places in the list to produce the following `ArrayList` as the result:

```
["It", "was", "a", "dark", "and", "stormy", "night"]
```

5. Write code to change the second element's value to "IS", producing the following `ArrayList` as the result:

```
["It", "IS", "a", "dark", "and", "stormy", "night"]
```

6. Write code to remove from the list any `Strings` that contain the letter "a". The following should be the list's contents after your code has executed:

```
["It", "IS", "stormy", "night"]
```

7. Write code to declare an `ArrayList` holding the first 10 multiples of 2: 0, 2, 4,..., 18. Use a loop to fill the list with the proper elements.
8. Write a method called `maxLength` that takes an `ArrayList` of `Strings` as a parameter and that returns the length of the longest `String` in the list. If your method is passed an empty `ArrayList`, it should return 0.
9. Write code to print out whether or not a list of `Strings` contains the value "IS". Do not use a loop.
10. Given the `ArrayList` from problem 4, write code to print out the index at which your list contains the value "stormy" and the index at which it contains "dark". Do not use a loop.

11. Given the `ArrayList` from problem 4, write a for-each loop that prints the uppercase version of each `String` in the list on its own line.
12. When the code that follows runs on an `ArrayList` of `String`s, it throws an exception. Why?

```
for (String s: words) {  
    System.out.println(s);  
    if (s.equals("hello")) {  
        words.add("goodbye");  
    }  
}
```

13. The code that follows does not compile. Why not? Explain how to fix it.

```
ArrayList<int> numbers = new ArrayList<>();  
numbers.add(7);  
numbers.add(19);  
System.out.println(numbers);
```

14. What is a wrapper class? Describe the difference between an `int` and an `Integer`.
15. Write the output produced when the following method is passed each of the following lists:

```

public static void mystery1(ArrayList<Integer> list) {
    for (int i = list.size() - 1; i > 0; i--) {
        if (list.get(i) < list.get(i - 1)) {
            int element = list.get(i);
            list.remove(i);
            list.add(0, element);
        }
    }
    System.out.println(list);
}

```

- a. [2, 6, 1, 8]
- b. [30, 20, 10, 60, 50, 40]
- c. [-4, 16, 9, 1, 64, 25, 36, 4, 49]

16. Write the output produced when the following method is passed each of the following lists:

```

public static void mystery2(ArrayList<Integer> list) {
    for (int i = list.size() - 1; i >= 0; i--) {
        if (i % 2 == 0) {
            list.add(list.get(i));
        } else {
            list.add(0, list.get(i));
        }
    }
}

```

```

    }

    System.out.println(list);
}

```

- a. [10, 20, 30]
- b. [8, 2, 9, 7, 4]
- c. [-1, 3, 28, 17, 9, 33]

17. Write the output produced when the following method is passed each of the following lists:

```

public static void mystery3(ArrayList<Integer> list) {
    for (int i = list.size() - 2; i > 0; i--) {
        int a = list.get(i);
        int b = list.get(i + 1);
        list.set(i, a + b);
    }
    System.out.println(list);
}

```

- a. [72, 20]
- b. [1, 2, 3, 4, 5, 6]
- c. [10, 20, 30, 40]

18. Write the output produced when the following method is passed each of the following lists:

```
public static void mystery4(ArrayList<Integer> list) {  
    for (int i = 0; i < list.size(); i++) {  
        int element = list.get(i);  
        list.remove(i);  
        list.add(0, element + 1);  
    }  
    System.out.println(list);  
}
```

- a. [10, 20, 30]
- b. [8, 2, 9, 7, 4]
- c. [-1, 3, 28, 17, 9, 33]

Section 10.2: The Comparable Interface

19. Describe how to arrange an `ArrayList` into sorted order. What must be true about the type of elements in the list in order to sort it?
20. What is a natural ordering? How do you define a natural ordering for a class you've written?
21. Consider the following variable declarations:

```
Integer n1 = 15;  
Integer n2 = 7;  
Integer n3 = 15;  
String s1 = "computer";  
String s2 = "soda";  
String s3 = "pencil";
```

Indicate whether the result of each of the following comparisons is positive, negative, or 0:

- a. `n1.compareTo(n2)`
- b. `n3.compareTo(n1)`
- c. `n2.compareTo(n1)`
- d. `s1.compareTo(s2)`
- e. `s3.compareTo(s1)`

f. `s2.compareTo(s2)`

22. Use the `compareTo` method to write code that reads two names from the console and prints the one that comes first in alphabetical order. For example, the program's output might look like the following:

```
Type a name: Tyler Durden
Type a name: Marla Singer
Marla Singer goes before Tyler Durden
```

23. Write code to read a line of input from the user and print the words of that line in sorted order, without removing duplicates. For example, the program output might look like the following:

```
Type a message to sort: to be or not to be that is the
question
Your message sorted: be be is not or question that the to
to
```

Exercises

1. Write a method called `averageVowels` that takes an `ArrayList` of strings as a parameter and returns the average number of vowel characters (a, e, i, o, u) in all Strings in the list. If your method is passed an empty `ArrayList`, it should return `0.0`.
2. Write a method called `swapPairs` that switches the order of values in an `ArrayList` of strings in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, then the next two, and so on. If the number of values in the list is odd, the method should not move the final element. For example, if the list initially stores `["to", "be", "or", "not", "to", "be", "hamlet"]`, your method should change the list's contents to `["be", "to", "not", "or", "be", "to", "hamlet"]`.
3. Write a method called `removeEvenLength` that takes an `ArrayList` of strings as a parameter and removes all of the strings of even length from the list.
4. Write a method called `doubleList` that takes an `ArrayList` of strings as a parameter and replaces every string with two of that same string. For example, if the list stores the values `["how", "are", "you?"]` before the method is called, it should store the values `["how", "how", "are", "are", "you?", "you?"]` after the method finishes executing.

5. Write a method called `scaleByK` that takes an `ArrayList` of integers as a parameter and replaces every integer of value k with k copies of itself. For example, if the list stores the values `[4, 1, 2, 0, 3]` before the method is called, it should store the values `[4, 4, 4, 4, 1, 2, 2, 3, 3, 3]` after the method finishes executing. Zeroes and negative numbers should be removed from the list by this method.
6. Write a method called `minToFront` that takes an `ArrayList` of integers as a parameter and moves the minimum value in the list to the front, otherwise preserving the order of the elements. For example, if a variable called `list` stores `[3, 8, 92, 4, 2, 17, 9]`, the value `2` is the minimum, so your method should modify the list to store the values `[2, 3, 8, 92, 4, 17, 9]`.
7. Write a method called `removeDuplicates` that takes as a parameter a sorted `ArrayList` of strings and eliminates any duplicates from the list. For example, if the list stores the values `["be", "be", "is", "not", "or", "question", "that", "the", "to", "to"]` before the method is called, it should store the values `["be", "is", "not", "or", "question", "that", "the", "to"]` after the method finishes executing. Because the values will be sorted, all of the duplicates will be grouped together. Assume that the `ArrayList` contains only `String` values, but keep in mind that it might be empty.
8. Write a method called `removeZeroes` that takes as a parameter an `ArrayList` of integers and eliminates any occurrences of

the number `0` from the list. For example, if the list stores the values `[0, 7, 2, 0, 0, 4, 0]` before the method is called, it should store the values `[7, 2, 4]` after the method finishes executing.

9. Write a method called `rangeBetweenZeroes` that takes as a parameter an `ArrayList` of integers and returns the number of indexes apart the two farthest occurrences of the number `0` are. For example, if the list stores the values `[7, 2, 0, 0, 4, 0, 9, 0, 6, 4, 8]` when the method is called, it should return `6`, because the occurrences of `0` that are farthest apart are at indexes 2 and 7, and the range 2 through 7 has six elements. If only one `0` occurs in the list, your method should return `1`. If no `0`s occur, your method should return `0`.
10. Write a method called `removeInRange` that accepts three parameters, an `ArrayList` of strings, a beginning string, and an ending string, and removes from the list any strings that fall alphabetically between the start and end strings. For example, if the method is passed a list containing the elements `["to", "be", "or", "not", "to", "be", "that", "is", "the", "question"]`, `"free"` as the start String, and `"rich"` as the end String, the list's elements should be changed to `["to", "be", "to", "be", "that", "the"]`. The `"or", "not", "is", and "question"` should be removed because they occur alphabetically between `" free"` and `" rich"`. You may assume that the start string alphabetically precedes the ending string.

11. Write a method called `stutter` that accepts an `ArrayList` of strings and an integer k as parameters and that replaces every string with k copies of that string. For example, if the list stores the values `["how", "are", "you?"]` before the method is called and k is 4, it should store the values `["how", "how", "how", "how", "are", "are", "are", "are", "you?", "you?", "you?", "you?"]` after the method finishes executing. If k is 0 or negative, the list should be empty after the call.
12. Write a method called `markLength4` that accepts an `ArrayList` of strings as a parameter and that places a string of four asterisks `"****"` in front of every string of length 4. For example, suppose that a variable called `list` contains the values `["this", "is", "lots", "of", "fun", "for", "Java", "coders"]`. The call of `markLength4(list);` should change the list to store the values `["*****", "this", "is", "*****", "lots", "of", "fun", "for", "*****", "Java", "coders"]`.
13. Write a method called `reverse3` that accepts an `ArrayList` of integer values as a parameter and reverses each successive sequence of three values in the list. If the list has extra values that are not part of a sequence of three, those values are unchanged. For example, if a list stores values `[3, 8, 19, 42, 7, 26, 19, -8]`, after the call the list should store the values `[19, 8, 3, 26, 7, 42, 19, -8]`. The first sequence of three `(3, 8, 19)` has been reversed to be `(19, 8, 3)`. The second sequence `(42, 7, 26)` has been reversed to be `(26, 7, 42)`,

and so on. Notice that 19 and -8 are unchanged because they were not part of a sequence of three values.

14. Write a method called `removeShorterStrings` that accepts an `ArrayList` of strings as a parameter and removes from each pair of values the shorter string in the pair. If the list is of odd length, the final element is unchanged. For example, suppose that a list contains `["four", "score", "and", "seven", "years", "ago", "our"]`. In the first pair (`"four"` and `"score"`) the shorter string is `"four"`. In the second pair (`"and"` and `"seven"`) the shorter string is `"and"`. In the third pair (`"years"` and `"ago"`) the shorter string is `"ago"`. Your method should remove these shorter strings, changing the list to store `["score", "seven", "years", "our"]`. If both strings in a pair have the same length, remove the first string in the pair.
15. Write a method called `filterRange` that accepts an `ArrayList` of integers and two integer values *min* and *max* as parameters and removes all elements whose values are in the range *min* through *max* (inclusive). For example, if a variable called `list` stores the values `[4, 7, 9, 2, 7, 7, 5, 3, 5, 1, 7, 8, 6, 7]`, the call of `filterRange(list, 5, 7);` should remove all values between 5 and 7, changing the list to store `[4, 9, 2, 3, 1, 8]`. If no elements in range *min-max* are found in the list, or if the list is initially empty, the list's contents are unchanged.

16. Write a method called `clump` that accepts an `ArrayList` of strings as a parameter and replaces each pair of strings with a single string that consists of the two original strings in parentheses separated by a space. If the list is of odd length, the final element is unchanged. For example, suppose that a list contains `["four", "score", "and", "seven", "years", "ago", "our"]`. Your method should change the list to store `["(four score)", "(and seven)", "(years ago)", "our"]`.
17. Write a method called `interleave` that accepts two `ArrayLists` of integers `a1` and `a2` as parameters and inserts the elements of `a2` into `a1` at alternating indexes. If the lists are of unequal length, the remaining elements of the longer list are left at the end of `a1`. For example, if `a1` stores `[10, 20, 30]` and `a2` stores `[4, 5, 6, 7, 8]`, the call of `interleave(a1, a2);` should change `a1` to store `[10, 4, 20, 5, 30, 6, 7, 8]`. If `a1` had stored `[10, 20, 30, 40, 50]` and `a2` had stored `[6, 7, 8]`, the call of `interleave(a1, a2);` would change `a1` to store `[10, 6, 20, 7, 30, 8, 40, 50]`.
18. Write a method called `mirror` that accepts an `ArrayList` of strings as a parameter and produces a mirrored copy of the list as output, with the original values followed by those same values in the opposite order. For example, if an `ArrayList` variable called `list` contains the values `["a", "b", "c"]`, after a call of `mirror(list);` it should contain `["a", "b", "c", "c", "b", "a"]`.

19. Modify the `Point` class from [Chapter 8](#) so that it defines a natural ordering by implementing the `Comparable` interface. Compare the `Points` by y -major order; that is, points with smaller y -coordinate values should come before those with higher y -coordinate values. Break ties by comparing x -coordinate values.
20. Modify the `TimeSpan` class from [Chapter 8](#) to include a `compareTo` method that compares time spans by their length. A time span that represents a shorter amount of time is considered to be “less than” one that represents a longer amount of time. For example, a span of 3 hours and 15 minutes is greater than a span of 1 hour and 40 minutes.
21. Modify the `CalendarDate` class from this chapter to include a year field, and modify its `compareTo` method to take years into account when making comparisons. Years take precedence over months, which take precedence over days. For example, July 18, 1995, comes before March 2, 2001.

Programming Projects

1. Write classes to model a shopping list. Make an `Item` class that represents a grocery item's name and price, such as tissues for \$3. Also implement an `ItemOrder` class that represents a shopper's desire to purchase a given item in a given quantity, such as five boxes of tissues. You might wish to implement bulk-discounted items, such as two boxes of tissues for \$4, which would bring the cost of the given item order of $2 + 2 + 1$ boxes of tissues to $\$4 + \$4 + \$3$, or \$11.00. Lastly, implement a `ShoppingCart` class that stores `ItemOrder`s in an `ArrayList` and allows item orders to be added to, removed from, or searched for in the cart. The cart should be able to report the total price of all item orders it currently carries.
2. Write a program to reverse the lines of a file and also to reverse the order of the words in each line of the file. Use `ArrayList`s to help you.
3. Write a family database program. Create a class to represent a person and to store references to the person's mother, father, and any children the person has. Read a file of names to initialize the name and parent-child relationships of each `Person`. (You might wish to create a file representing your own family tree.) Store the overall list of `Person`s as an `ArrayList`.

Write an overall main user interface that asks for a name and prints the maternal and paternal family line for that person. Here's a hypothetical execution of the program, using as an input file the line of English Tudor monarchs:

```
Person's name? Henry VIII
```

```
Maternal line:
```

```
    Henry VIII
```

```
        Elizabeth of York
```

```
Paternal line:
```

```
    Henry VIII
```

```
        Henry VII
```

```
Children:
```

```
    Mary I
```

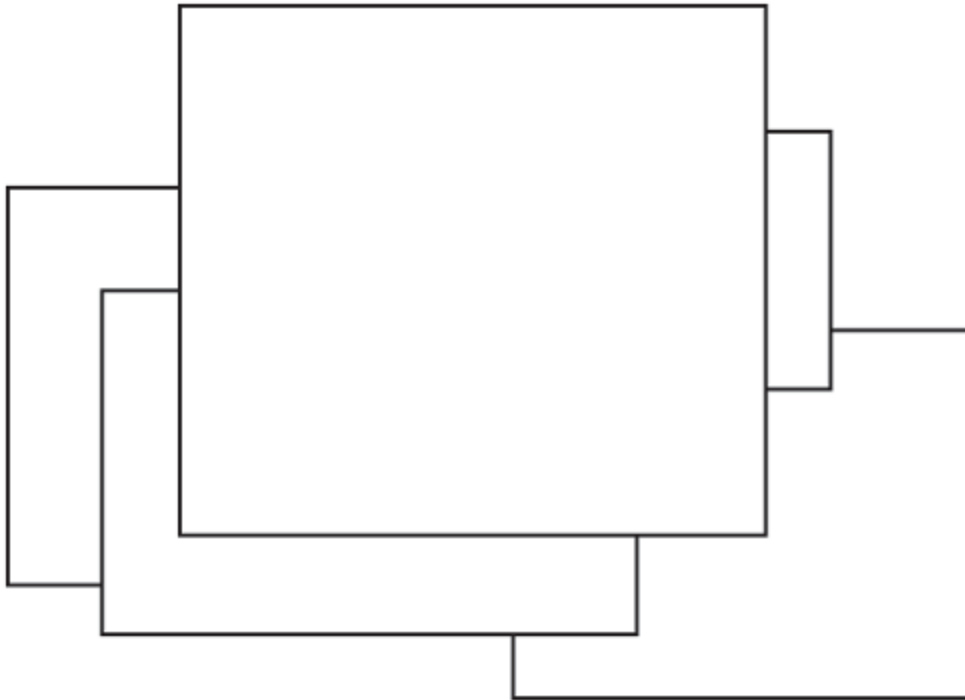
```
    Elizabeth I
```

```
    Edward VI
```

4. Write a class that models a list of possibly overlapping rectangular two-dimensional window regions, like the windows for the programs open on your computer. The order of the rectangles in the list implies the order in which they would display on the screen (sometimes called the “*z*-order”), from 0 on the bottom to `size() - 1` on the top.

Each rectangle stores its (x, y) position, width, and height. Your rectangle list class should have a method that takes a `Point` as a parameter, treats it as though the user clicked that

`Point` on the screen, and moves the topmost rectangle touching that `Point` to the front of the list.



5. Extend the ranked choice voting case study to allow for incomplete preferences. Voters are normally allowed to fill out the ballot in an incomplete manner. For example, with the sample ballots with four candidates, you might have ballots that specify only a first choice or only two choices. When you have partial information, you can end up with empty ballots because the voter might have chosen candidates who are all eliminated.

Modify the `Ballot` class to have an `isEmpty` method that indicates that there are no candidates left. (Modify the `getCandidate` method to return `"none"` in this case.) Then modify the `TallyVotes` program to account for potentially

empty ballots. It is possible that the input file will contain blank lines because a voter didn't pick any candidates, so include a test that ignores blank lines in the input file. Modify the code that eliminates candidates so that it removes from the list of ballots any ballots that become empty. That way the percentages will be reported correctly on the next round of tallying.